

# Práctica: Sistema de Conteo Automático con Visión Artificial y YOLO

**Objetivo:** Desarrollar un sistema capaz de detectar y contar objetos utilizando redes neuronales convolucionales (YOLO) y Python.

## Contexto:

El conteo automático de objetos es una de las aplicaciones más demandadas en la industria. Permite automatizar inventarios, controlar el tráfico vehicular, monitorizar aforos de personas o realizar seguimiento en líneas de producción.

---

## Paso 1: Preparación del Entorno

Utilizaremos la librería ultralytics, que contiene la implementación de YOLO.

```
# Instalación de la librería Ultralytics (YOLO)
!pip install ultralytics
```

---

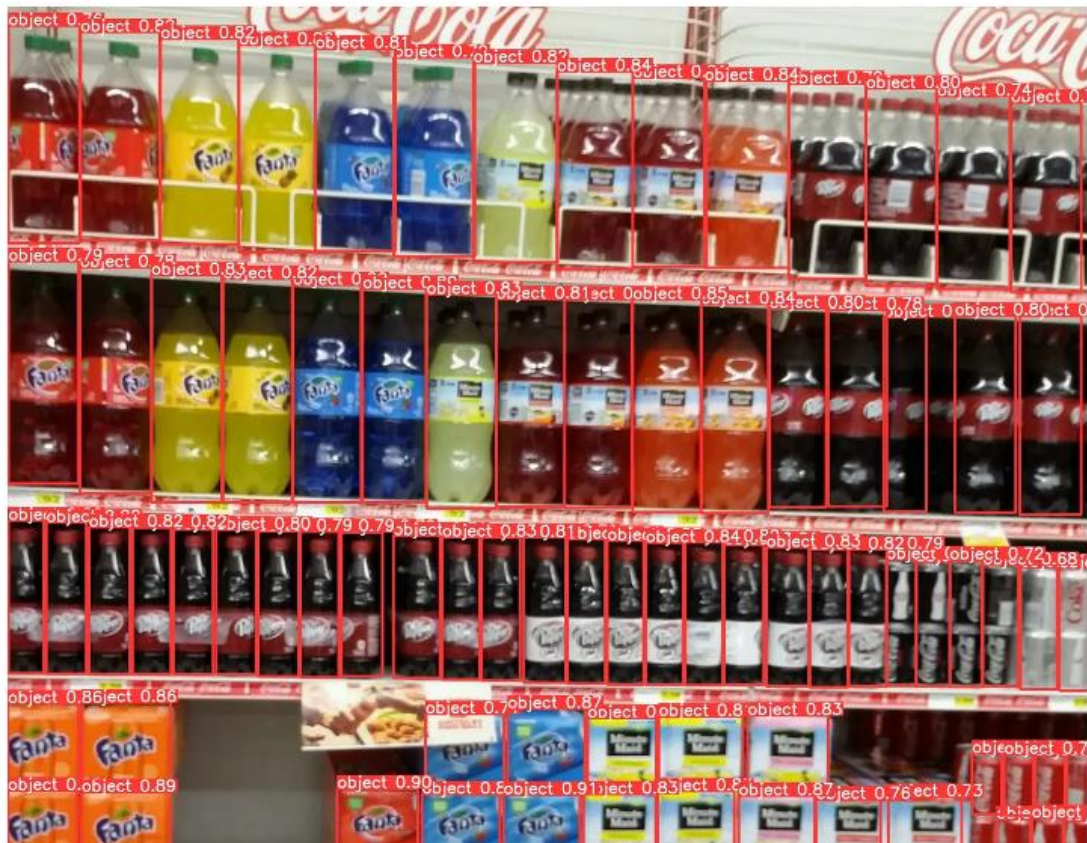
## Paso 2: Obtención de los Datos (Dataset)

Para que una IA aprenda a detectar objetos, necesita ejemplos. Usaremos un dataset llamado "bbox-retail" que contiene imágenes de estanterías con sus productos ya etiquetados.

Este dataset solo tiene una clase de objetos (0) y por tanto no diferencia unos productos de otros.

**Instrucción:** Descarga y prepara el dataset

<https://universe.roboflow.com/try1-bxa32/bbox-retail-tdzdt>



---

## Paso 3: Carga del Modelo de IA

Carga el modelo y entrénalo con el dataset seleccionado.

---

## Primera Detección (Inferencia)

Vamos a probar el modelo con una imagen que nunca ha visto (del set de prueba) para ver qué detecta.

**Instrucción:** Ejecuta la detección en una imagen aleatoria.

```
# Realizar predicción sobre una imagen específica
img_path = 'ruta_a_la_imagen.jpg'
resultados = model(img_path)
```

```
# Mostrar la cantidad bruta de detecciones
print(f"Objetos detectados inicialmente: {len(resultados[0].boxes)}")
```

Probablemente verás un número alto (ej. 60+ objetos). Sin embargo, no todas esas detecciones son correctas. Algunas son "ruido".

---

## Paso 5: Filtrado y Lógica de Conteo

Aquí es donde aplicamos la lógica de programación. El modelo nos devuelve muchas cajas ("bounding boxes"), cada una con un nivel de **Confianza** (**Confidence Score**) entre 0 y 1.

Si la confianza es baja (ej. 0.2), es probable que el modelo se esté equivocando. Vamos a filtrar para quedarnos solo con las detecciones seguras.

### ¿Qué herramienta usamos?

Counter de la librería collections es una herramienta muy potente de Python. Toma una lista (ej: [0, 0, 1, 0]) y devuelve un diccionario con las cuentas (ej: {0: 3, 1: 1}).

**Instrucción:** Implementa el filtro de confianza y el contador.

```
from collections import Counter
```

```
# 1. Definir un umbral de confianza (Tú decides qué tan estricto ser)
UMBRAL_CONF = 0.4 # Solo aceptamos detecciones con >40% de seguridad
```

```
# 2. Extraer las cajas de la predicción
```

```
prediccion = resultados[0]
cajas_totales = prediccion.boxes
```

```
# 3. Filtrar usando una List Comprehension
```

```
# Nos quedamos con la caja SOLO SI su confianza (box.conf) es mayor al umbral
boxes_filtradas = [box for box in cajas_totales if box.conf > UMBRAL_CONF]
```

```
print(f"Objetos reales tras filtrar (Confianza > {UMBRAL_CONF}):
{len(boxes_filtradas)}")
```

```
# 4. Realizar el conteo por tipo de objeto
```

```
# box.cls nos dice qué clase de objeto es (0, 1, 2...).
ids_clases = [int(box.cls) for box in boxes_filtradas]
```

```
conteos = Counter(ids_clases)

print("\n REPORTE DE INVENTARIO:")
nombres_clases = model.names
for clase_id, cantidad in conteos.items():
    nombre_obj = nombres_clases[clase_id]
    print(f" -> {nombre_obj}: {cantidad} unidades")
```

---

## Paso 6: Visualización de Resultados

Un sistema de visión artificial no está completo si no podemos visualizar lo que la máquina "ve".

Usaremos **OpenCV** y **Matplotlib** para dibujar las cajas y escribir el conteo en la imagen.

**Instrucción:** Crea la función de visualización.

```
import cv2
import matplotlib.pyplot as plt

def visualizar_conteo(prediccion, boxes_filtradas, conteos, umbral):
    # 1. Filtrar visualmente las cajas en el objeto de predicción
    # (Esto actualiza la imagen interna para mostrar solo lo que pasó el
    filtro)
    prediccion.bboxes = prediccion.bboxes[prediccion.bboxes.conf > umbral]

    # 2. Generar la imagen con las cajas dibujadas (array numpy)
    img_resultado = prediccion.plot()

    # 3. Crear el texto para la anotación
    nombres = prediccion.names
    texto_conteo = ", ".join([f"{nombres[c]}: {n}" for c, n in
    conteos.items()])

    # 4. Escribir el texto en la imagen (usando OpenCV)
    # (Imagen, Texto, Coordenadas (x,y), Fuente, Tamaño, Color (BGR), Grosor)
    cv2.putText(img_resultado, f"Total: {len(boxes_filtradas)}", (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
```

```
# 5. Convertir de BGR (formato OpenCV) a RGB (formato Matplotlib)
```

"OpenCV lee los colores al revés (Azul-Verde-Rojo). Si no los ordenamos correctamente (Rojo-Verde-Azul) antes de mostrar la imagen en pantalla con Matplotlib, los objetos rojos se verán azules y viceversa."

```
img_rgb = cv2.cvtColor(img_resultado, cv2.COLOR_BGR2RGB)
```

```
# 6. Mostrar
plt.figure(figsize=(12, 8))
plt.imshow(img_rgb)
plt.axis('off')
plt.title(f"Detección con Umbral: {umbral}")
plt.show()
```

```
# ¡Llamamos a la función!
```

```
visualizar_conteo(resultados[0], boxes_filtradas, conteos, UMBRAL_CONF)
```

---

## Entregable

Ahora que tienes el código base, automatiza el proceso.

1. Crea una función llamada `analizar_estanteria(ruta_imagen)` que reciba una imagen y devuelva el número total de productos.
2. Utiliza el siguiente código para obtener todas las imágenes de la carpeta de prueba (test).
3. Ejecuta tu función en un bucle y muestra el conteo de las primeras 3 imágenes.

```
# Pista para listar imágenes
```

`glob.glob(...)` devuelve una **LISTA** de Python (un array) llena de cadenas de texto (strings) con las rutas completas.

```
import glob
```

```
rutas_test = glob.glob('/content/datasets/test/images/*.jpg')
```

```
# TU CÓDIGO AQUÍ:
```

```
# for ruta in rutas_test[:3]:
```

```
#     ...
```

4. Al finalizar, el alumno deberá mostrar una captura de pantalla con una imagen procesada donde se vean los recuadros (bounding boxes) y el texto con el conteo superpuesto correctamente.

5. Todo debe ir incluido en un documento PDF donde se muestre la salida de cada instrucción y su interpretación, ejemplo valores como cls-> clase, conf-> valor de confianza...

**Continuación, segunda parte.**

6. Realiza de nuevo la práctica anterior, pero en este caso eligiendo un dataset donde haya más de una clase de objetos, realizando el conteo de cada uno de ellos en cada imagen.